UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE
Department of Computer Science
Software Engineering
36862 – Object-Oriented Programming

UNIT 1 PROJECT · TEAM WORK

List of Possible Objects: ProvStock

Prov&Tec Management, a Comprehensive Sales and Inventory System

Team	ByteWorks
Members	Andrés Pachacama, Carlos Parra, Matias Paucar
NRC	36862
Instructor	Edison Lascano, PhD
Date	October 26, 2026
Academic period	SII-2026 (September 2026 – February 2027)
Version	1.2

Revision History

Table 1
Document Versions

Version	Date	Team	Author	Description of Changes
1.0	October 13, 2026	Team ByteWorks	Carlos Parra	Initial list of the possible objects of ProvStock: candidate nouns and, for each class of the UML Design Specification, its attributes, responsibilities, and example objects taken from the sample data.
1.1	October 20, 2026	Team ByteWorks	Andrés Pachacama	Persistence and support objects updated to the program version 1.4 (Table 6): data saved in JSON files, reports exported to CSV, and console menu divided into one class per module (clean code).
1.2	October 26, 2026	Team ByteWorks	Matias Paucar	Classes of the last clean code review added: ProductInfo and SaleFilter (Section 3.6) and CsvReportExporter (Table 6); references updated to the Project Definition v1.2, the SRS v1.3, and the UML Design Specification v1.3.

1. Introduction

1.1 Purpose

This document lists the possible objects of ProvStock, the sales and inventory management system for Prov&Tec. For each object, it explains what the object knows (its attributes), what it does (its methods), and gives real examples taken from the program. It is the link between the problem described in the Project Definition v1.2 and the class diagram of the UML Design Specification v1.3.

1.2 Method

The objects were found with a noun analysis of the problem, following these steps:

- The problem statement (P1 to P7), the requirements of the Software Requirements Specification (SRS) v1.3, and the use cases were read, and their nouns were written down.
- A noun became an **object** when the system has to remember it and it has its own data and behavior; for example, a sale or a product.

- A noun became an **attribute** when it only describes another object; for example, the price describes a product.
- A noun was **discarded** when it describes the current manual situation or something outside the scope of this version; for example, paper or electronic invoicing.
- The objects with the same data and behavior were grouped into a class, and some **control objects** were added in the design to coordinate each module.

1.3 Conventions

An object is a concrete thing that the system handles, such as the product CAB-001; a class is the template from which those objects are created, such as Product. Class, attribute, and method names are written in English, following the Java naming conventions (NFR-05), because they are the names used in the code. The requirement identifiers (P, FR, BR) refer to the Project Definition v1.2 and the SRS v1.3.

2. Candidate Nouns

Table 2 shows the nouns found in the problem and the requirements, where they appear, and the decision taken with each one.

Table 2
Nouns of the Problem and Decision Taken

Noun	Where it appears	Decision	Result
User, staff	P7, FR-01	Object	User
Role (salesperson, warehouse clerk, owner)	P7, FR-01.1	Object	Role
Password	FR-01.2, NFR-02	Attribute	User.passwordHash, User.salt
Customer, buyer	P1, FR-02	Object	Customer
Final Consumer	FR-02.5	Object (special instance)	Customer 9999999999999

Noun	Where it appears	Decision	Result
Cédula, RUC, passport	FR-02.2, Table B1	Attribute	Customer.identificationType, identificationNumber
Supplier	P1, FR-06	Object	Supplier
Product, electrical material	P1, FR-03	Object	Product
Category, brand, unit of measure	Table B2	Attribute	Product.category, brand, unitOfMeasure
Location in the warehouse	P2, FR-03.4	Attribute	Product.location
Price, cost	FR-03.2	Attribute	Product.salePrice, purchaseCost
Stock, minimum stock level	P2, P5, FR-03.1	Attribute	Product.currentStock, minimumStock
Sale, sales slip	P4, FR-04	Object	Sale
Line of a sale	FR-04.3	Object	SaleDetail
VAT rate, payment method	BR-05, FR-04.6	Attribute	Sale.vatRate, paymentMethod
Void reason	FR-04.8	Attribute	Sale.voidReason, voidDateTime, voidedBy
Purchase, order form	P3, FR-06.3	Object	Purchase
Line of a purchase	FR-06.3	Object	PurchaseDetail
Supplier invoice number	FR-06.3	Attribute	Purchase.supplierInvoiceNumber
Incoming and outgoing goods, stock adjustment	P2, P7, FR-05.6	Object	StockMovement
Inventory	P2, FR-05	Control object	Inventory
Alert, stockout	P5, FR-08	Calculated, not stored	StockMonitor
Sales history	P6, FR-07	Control object	SalesManager
Report (sales, inventory, purchases, slow-moving products)	P6, FR-09	Object	SalesReport, InventoryReport, PurchaseReport, SlowMovingProductsReport
Paper, spreadsheet	Problem	Discarded	Current manual situation
Company, store, warehouse, market	Problem	Discarded	Context of the system

Noun	Where it appears	Decision	Result
Electronic invoice, SRI, branch	Background, scope	Discarded	Outside the scope of this version

Note. The alert is not stored as an object: it is calculated each time from the stock and the minimum stock of the product, so it disappears by itself when the stock rises (FR-08.4).

3. Possible Objects

3.1 Summary

Table 3 lists the 26 possible objects of ProvStock grouped in five types: domain objects, which are the business things that the system stores; report objects, which summarize the data; control objects, which coordinate each module; persistence objects, which save the data and the reports; and argument objects, which group the data that one object gives to another.

Table 3
Possible Objects of ProvStock

Nº	Object (class)	Type	Description	Backlog
1	Role	Domain	One of the three roles and its permissions.	RF01
2	User	Domain	A person with an account in the system.	RF01
3	Customer	Domain	A person or company that buys.	RF02
4	Supplier	Domain	A company from which the business buys.	RF06
5	Product	Domain	An electrical material that is sold.	RF03
6	Sale	Domain	A sale to a customer.	RF04
7	SaleDetail	Domain	One line of a sale.	RF04
8	Purchase	Domain	Goods bought from a supplier.	RF06
9	PurchaseDetail	Domain	One line of a purchase.	RF06
10	StockMovement	Domain	Any change in the stock of a product.	RF05
11	SalesReport	Report	Sales of a period.	RF09

Nº	Object (class)	Type	Description	Backlog
12	InventoryReport	Report	Stock and its value.	RF09
13	PurchaseReport	Report	Purchases per supplier in a period.	RF09
14	SlowMovingProductsReport	Report	Products without sales in a period.	RF09
15	AccessManager	Control	Sessions, permissions, and user accounts.	RF01
16	CustomerManager	Control	The list of customers.	RF02
17	ProductCatalog	Control	The list of products.	RF03
18	SalesManager	Control	Sales, VAT rate, and sales history.	RF04, RF07
19	Inventory	Control	Every change of the stock.	RF05
20	PurchasingManager	Control	Suppliers and purchases.	RF06
21	StockMonitor	Control	The stock alerts.	RF08
22	ReportGenerator	Control	Creates the four reports.	RF09
23	FileManager	Persistence	Saves and loads all the data.	SRS 2.4
24	CsvReportExporter	Persistence	Saves each report as a CSV file.	RF09
25	ProductInfo	Argument	The editable data of a product.	RF03
26	SaleFilter	Argument	The criteria to filter the sales.	RF07

3.2 Domain Objects

The domain objects are the “things” of the business that the system must remember. Each card shows what the object knows, what it does, and real examples created by the program with its sample data.

1. ROLE	
Description	One of the three fixed roles of the system, with the list of permissions that it gives (Table 10 of the UML Design Specification).
What it knows	id, name, permissions
What it does	hasPermission(permission): says whether the role allows an operation.
Example objects	SALESPERSON: VIEW_PRODUCTS, MANAGE_CUSTOMERS, REGISTER_SALE. WAREHOUSE_CLERK: VIEW_PRODUCTS, MANAGE_PRODUCTS, VIEW_INVENTORY, ADJUST_STOCK, REGISTER_PURCHASE, RECEIVE_ALERTS. OWNER: all 15 permissions.
Source	P7, FR-01.1

2. USER	
Description	A person of the staff who has an account to use the system.
What it knows	id, username, fullName, passwordHash, salt, role, active
What it does	verifyPassword(), changePassword(), resetPassword(), hasPermission(), deactivate().
Example objects	owner – Propietario (OWNER). vendedor – María López (SALESPERSON). bodeguero – Pedro Ramírez (WAREHOUSE_CLERK).
Source	P7, FR-01, Table B6

3. CUSTOMER	
Description	A person or company that buys from Prov&Tec. The Final Consumer is a special customer for buyers who do not give their data.
What it knows	id, identificationType, identificationNumber, name, phone, email, address, active
What it does	validate(), validateIdentification(), isFinalConsumer(), update(), deactivate().
Example objects	CONSUMIDOR FINAL – RUC 9999999999999 (cannot be modified or deactivated). Juan Pérez – cédula 1712345675, Quito. Instalaciones Andes – RUC 1701234567001, Sangolquí.
Source	P1, FR-02, Table B1

4. SUPPLIER	
Description	A company from which Prov&Tec buys the products that it sells.
What it knows	id, ruc, name, contactPerson, phone, email, address, active
What it does	validate(), validateRuc(), update(), deactivate().
Example objects	Electrocables S.A. – RUC 1790012345001, contact Ana Torres. Distribuidora Eléctrica del Pacífico – RUC 0992345678001, contact Luis Mora.
Source	P1, FR-06, Table B3

5. PRODUCT	
Description	An electrical material sold by the company. It keeps its stock and its place in the warehouse.
What it knows	id, code, name, description, category, brand, unitOfMeasure, location, purchaseCost, salePrice, currentStock, minimumStock, active
What it does	hasStock(), isLowStock(), getStockValue(), updateStock() (called only by Inventory), update(info), getInfo(), deactivate(). It receives its editable data in a ProductInfo object.
Example objects	CAB-001 – Cable THHN 12 AWG, meter, Pasillo A1, cost $0.45, price $0.70, minimum 100. ILU-001 – Foco LED 9W luz blanca, unit, Pasillo B1, price $2.50. PRO-001 – Breaker 1 polo 20A, stock 8 and minimum 10 (it has a stock alert).
Source	P1, P2, P5, FR-03, Table B2

6. SALE	
Description	A sale made by a salesperson to a customer. It replaces the paper sales slip and keeps the VAT rate of the day.
What it knows	number, dateTime, registeredBy, customer, details, paymentMethod, vatRate, subtotal, vatAmount, total, status, voidReason, voidDateTime, voidedBy
What it does	addProduct(), removeProduct(), calculateSubtotal(), calculateVat(), calculateTotal(), voidSale(), isVoided(), validate().
Example objects	Sale Nº 1 – Final Consumer, María López, CASH, VAT 15%: subtotal $10.00, VAT $1.50, total $11.50, REGISTERED. The same sale after it is voided: status VOIDED, reason “Error de digitación”, voided by owner.
Source	P4, FR-04, Table B4

7. SALEDETAIL	
Description	One line of a sale: a product, its quantity, and the price of the day. It cannot exist without its sale (composition).
What it knows	product, quantity, unitPrice
What it does	calculateSubtotal(), validate().
Example objects	ILU-001 × 4 units × $2.50 = $10.00 (line of sale Nº 1).
Source	FR-04.3

8. PURCHASE	
Description	The goods received from a supplier. It replaces the paper order form and increases the stock.
What it knows	number, dateTime, registeredBy, supplier, supplierInvoiceNumber, details, total
What it does	addProduct(), removeProduct(), calculateTotal(), validate().
Example objects	Purchase Nº 1 – Electrocables S.A., invoice 001-001-000123, registered by bodeguero, total $17.50.
Source	P3, FR-06, Table B5

9. PURCHASEDETAIL	
Description	One line of a purchase: a product, its quantity, and its unit cost (composition with Purchase).
What it knows	product, quantity, unitCost
What it does	calculateSubtotal(), validate().
Example objects	CAB-002 × 50 meters × $0.35 = $17.50 (line of purchase Nº 1).
Source	FR-06.3

10. STOCKMOVEMENT	
Description	Any change in the stock of a product, with who made it and why. The stock of a product is always the sum of its movements.
What it knows	id, dateTime, product, type, quantity, sale, purchase, user, reason
What it does	validate().
Example objects	ADJUSTMENT +500 of CAB-001 by owner, reason “Inventario inicial”. SALE −4 of ILU-001 by vendedor, Venta Nº 1. PURCHASE +50 of CAB-002 by bodeguero, Compra Nº 1. VOIDED_SALE +4 of ILU-001 by owner, Venta Nº 1.
Source	P2, P7, FR-05.6, Table B7

3.3 Report Objects

The report objects are created by ReportGenerator when the owner asks for a report (UC-19). They do not change any data: they only read the sales, purchases, and products of a period and calculate the totals. Table 4 describes them.

Table 4
Report Objects

Object	What it knows	What it does	Example
SalesReport	startDate, endDate, generatedAt, sales	getSalesCount(), getTotalSold(), getVatCollected(), getTotalBySalesperson(), getBestSellingProducts()	October 2026: 2 sales, $19.55 sold, $2.55 of VAT.
InventoryReport	generatedAt, products	getTotalStockValue()	6 products with a stock value of $477.00.
PurchaseReport	startDate, endDate, generatedAt, purchases	getTotalCost(), getTotalBySupplier()	Electrocables S.A.: $17.50.
SlowMovingProductsReport	startDate, endDate, generatedAt, products, sales	getProductsWithoutSales(), getImmobilizedValue()	Active products with no sales in the period and the money they keep stopped.

Note. All four reports also have generate(). They do not write files: CsvReportExporter saves them as CSV files (Table 6). In version 2 of the class diagram, they will share an abstract superclass Report.

3.4 Control Objects

The control objects do not come from a noun of the problem. They were added in the design so that each Product Backlog item has one object that keeps its collection and applies its rules (Table 5). Only one object of each control class exists while the program is running.

Table 5
Control Objects

Object	What it keeps	What it does	Use cases
AccessManager	users, roles, currentUser	Opens and closes sessions, checks permissions, and creates, updates, and deactivates accounts.	UC-01 to UC-04
CustomerManager	customers	Registers, updates, searches for, and deactivates customers; creates the Final Consumer.	UC-05
ProductCatalog	products	Registers, updates, searches for, and deactivates products.	UC-06, UC-07
SalesManager	sales, nextSaleNumber, vatRate	Registers and voids sales, keeps the VAT rate, and filters the sales history with a SaleFilter.	UC-08 to UC-10, UC-16
Inventory	movements	Is the only object that changes the stock; records every movement and makes the adjustments.	UC-11 to UC-13
PurchasingManager	suppliers, purchases, nextPurchaseNumber	Manages the suppliers and registers the purchases.	UC-14, UC-15
StockMonitor	productCatalog (reference)	Finds the active products at or below their minimum stock and counts the alerts.	UC-17, UC-18
ReportGenerator	salesManager, purchasingManager, productCatalog, fileManager (references)	Reads the JSON files again and creates the four report objects.	UC-19, UC-20

3.5 Persistence and Support Objects

FileManager saves the collections of the control objects in JSON files (one file per class, with the Gson library) after every operation and loads them again when the program opens, and CsvReportExporter saves each report as a CSV file; both appear in the class

diagram. The other classes of Table 6 are in the code but not in the class diagram, because they are neither part of the business nor of the control: they form the console interface and the utilities.

Table 6
Persistence and Support Objects

Object	Package	Responsibility
FileManager	persistence	Saves all the data in JSON files (user.json, product.json, sale.json, etc.) and loads it again at startup.
CsvReportExporter	persistence	Saves each of the four reports as a CSV file in the folder reports, for example reporte_inventario_20261026_134711.csv (UC-20).
ProvStockApp	view	Starts the program: loads the data, shows the welcome screen, and logs the user in (UC-01).
AppContext	view	Creates the control objects once and gives them to the menus.
MainMenu	view	Shows only the options allowed for the role of the user (UI-02).
UserMenu, CustomerMenu, ProductMenu, SalesMenu, InventoryMenu, PurchaseMenu, ReportMenu	view	One menu per module; each one runs the use cases of its Product Backlog item.
ConsoleHelper, TextFormat	view	Shared behavior of the menus: submenus, error handling, product lists, and text formats.
SampleData	view	Creates the sample data the first time the program runs.
InputReader	view	Reads the data typed on the keyboard and asks again when a value is not valid.
IdentificationValidator	utils	Validates the cédula, the RUC, the passport, the phone, and the email.
CsvWriter	utils	Writes the rows of a CSV file, separated by semicolons; CsvReportExporter uses it.

3.6 Argument Objects

The argument objects were added in the clean code review. Instead of passing many separate values to a method, the program puts them in one object (Martin, 2008). They are created, used once, and discarded; they are not saved in the files. Table 7 describes them.

Table 7
Argument Objects

Object	What it knows	What it does	Example
ProductInfo	name, category, unitOfMeasure, description, brand, location, purchaseCost, salePrice, minimumStock	withDescription(), withBrand(), withLocation(), withPrices(), withMinimumStock(): each one fills a value and returns the same object.	Tomacorriente doble, SWITCHES_AND_OUTLETS, UNIT, brand Veto, Pasillo C2, cost $1.30, price $2.40, minimum 10.
SaleFilter	startDate, endDate, salesperson, customer, status	from(), to(), bySalesperson(), byCustomer(), withStatus(); matches(sale) says whether a sale meets all the criteria.	Sales of vendedor from 2026-10-01 to 2026-10-31 (sales history); the same dates without salesperson (sales report).

Note. A criterion that is not given is not applied: a SaleFilter with no criteria selects every sale.

4. Relationships between the Objects

The objects do not work alone: a sale knows its customer and its salesperson, and each line of the sale knows its product. Table 8 shows the main relationships with an example taken from the sample data.

Table 8
Main Relationships between the Objects

Object	Relationship	Object	Example
User	acts as (many to 1)	Role	vendedor acts as SALESPERSON.
Customer	makes (1 to many)	Sale	CONSUMIDOR FINAL makes sale Nº 1.
User	registers (1 to many)	Sale	María López registers sale Nº 1.
Sale	is composed of (1 to 1..*)	SaleDetail	Sale Nº 1 has the line ILU-001 × 4.

Object	Relationship	Object	Example
SaleDetail	refers to (many to 1)	Product	The line refers to ILU-001.
Supplier	supplies (1 to many)	Purchase	Electrocables S.A. supplies purchase Nº 1.
Purchase	is composed of (1 to 1..*)	PurchaseDetail	Purchase Nº 1 has the line CAB-002 × 50.
Product	records (1 to many)	StockMovement	CAB-001 records its initial inventory of 500 m.
StockMovement	originated by (0..1)	Sale or Purchase	SALE −4 is originated by sale Nº 1.
StockMovement	performed by (many to 1)	User	The adjustment of CAB-001 was performed by owner.
Product	uses	ProductInfo	CAB-001 is created from a ProductInfo with its name, category, unit, prices, and location.
SaleFilter	selects	Sale	The filter from 2026-10-01 to 2026-10-31 selects the sales of October.

Note. The complete list of relationships and multiplicities is in Tables 7 and 8 of the UML Design Specification v1.3.

5. Traceability

Table 9 relates each Product Backlog item to the problems that it solves and to the objects that implement it. Every problem of the Project Definition is covered by at least one object, and every object is justified by a requirement.

Table 9
Traceability between Problems, Backlog Items, and Objects

Backlog	Module	Solves	Objects
RF01	Access Management	P7	AccessManager, User, Role
RF02	Customer Management	P1, P4	CustomerManager, Customer
RF03	Product Management	P1, P2, P4	ProductCatalog, Product, ProductInfo
RF04	Sales Management	P2, P4, P7	SalesManager, Sale, SaleDetail
RF05	Inventory Management	P2, P3, P7	Inventory, StockMovement, Product

Backlog	Module	Solves	Objects
RF06	Supplier and Purchase Management	P1, P2, P7	PurchasingManager, Supplier, Purchase, PurchaseDetail
RF07	Sales History	P6, P7	SalesManager, Sale, SaleFilter
RF08	Stock Alerts	P3, P5	StockMonitor, Product
RF09	Reports	P3, P6	ReportGenerator, SalesReport, InventoryReport, PurchaseReport, SlowMovingProductsReport, CsvReportExporter
—	Data persistence	P1	FileManager

References

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Pachacama, A., Parra, C., & Paucar, M. (2026a). *Project definition: ProvStock* (Version 1.2) [Unpublished manuscript]. Department of Computer Science, Universidad de las Fuerzas Armadas ESPE.

Pachacama, A., Parra, C., & Paucar, M. (2026b). *Software requirements specification: ProvStock* (Version 1.3) [Unpublished manuscript]. Department of Computer Science, Universidad de las Fuerzas Armadas ESPE.

Pachacama, A., Parra, C., & Paucar, M. (2026c). *UML design specification: ProvStock* (Version 1.3) [Unpublished manuscript]. Department of Computer Science, Universidad de las Fuerzas Armadas ESPE.